

Practice 2: Thread

Redes para Sistemas Embebidos



ITESO

**Universidad Jesuita
de Guadalajara**

Team 7:

Sofia Davis Rodríguez

Cuauhtemoc Alejandro Herrera Pérez

Plinio Yadin Morales Rodriguez

Professor: Estephania Martínez García

Data: 23 de Septiembre, 2026

Content

Practice 2: Thread	1
Introduction	3
Repository link:	3
Thread concepts used	4
IEEE 802.15.4	4
6LoWPAN and IPv6.....	4
Mesh networking.....	4
Thread device roles	4
Network formation and joining	4
Network identifiers and settings.....	4
Thread security.....	4
Self-healing behavior	4
Low-power communication	5
IPv6 addressing	5
Border Router.....	5
CoAP	5
CoAP methods.....	5
CoAP message types	5
Multicast communication.....	5
Detailed network topology of the final setup.....	6
Thread Network Settings.....	6
Development	7
Modifications made.....	7
Screenshots.....	13
Conclusions	15

Introduction

Team 7 — Channel 11, PAN ID 0x7777

For this practice we worked with Thread, a low-power wireless mesh networking protocol built on top of IPv6 and 802.15.4, running on FRDM-KW41Z boards. Thread networks are self-healing and self-forming, meaning devices can automatically discover each other, pick a channel, generate a PAN ID, and organize themselves into roles (Leader, Router, REED) without needing a central coordinator to configure everything by hand. On top of this network layer, we used CoAP (Constrained Application Protocol), a lightweight, UDP-based protocol designed specifically for small, embedded devices using methods like GET, POST, PUT and DELETE through simple URIs.

The practice consisted of extending a base CoAP demo application so that one device (the Leader) runs a shared counter and an accelerometer reading service, while a second router-eligible device (Router2) periodically polls the Leader for both values, alternating between confirmable (CON) and non-confirmable (NON) CoAP requests. We also added custom URIs to control the counter remotely (start/stop/restart/change interval). This gave us hands-on experience with how a mesh network actually behaves at the application layer, discovering peers, handling message loss, and reacting to sensor data in near real time.



Repository link:

https://github.com/Cuau-herrera/P2_Redex

Thread concepts used

IEEE 802.15.4

Thread uses this wireless standard for communication between nearby devices. It is designed for low-power, low-data-rate networks, which makes it suitable for IoT devices such as sensors and controllers.

6LoWPAN and IPv6

Thread carries IPv6 packets over IEEE 802.15.4. Since radio frames are small, 6LoWPAN helps adapt IPv6 traffic to the link by compressing headers and supporting packet fragmentation when needed.

Mesh networking

Thread devices can forward traffic for other devices. This allows information to travel across multiple nodes instead of requiring every device to communicate directly with every other device.

Thread device roles

A Leader manages important network information and helps coordinate the network. Routers forward traffic, while End Devices communicate through a parent, usually a Router. A REED (Router Eligible End Device) can act as an End Device or become a Router if the network requires it. The Leader is selected from the network's Router-capable devices; it is not necessarily a permanent central controller.

Network formation and joining

A Thread network can be formed by a device, and other devices can discover and join it if they have the required credentials and compatible settings. Commissioning is the process used to securely authorize a device to join.

Network identifiers and settings

The Network Name is a human-readable name. The PAN ID identifies the 802.15.4 personal-area network, and the Extended PAN ID identifies the Thread network. The channel specifies which radio frequency the network uses. Devices need compatible network information to communicate.

Thread security

Thread uses authentication and encryption to protect network communication. A device must be authorized to join, and messages are protected using network security keys. Commissioning helps provide network credentials securely.

Self-healing behavior

Because Thread uses a mesh, the network can adapt when a device becomes unavailable or a route changes. Routers can update paths so traffic may continue through other available nodes.

Low-power communication

Thread is designed for devices that may need to operate for long periods on limited power. End Devices can reduce their radio activity and rely on a parent Router to relay messages when needed.

IPv6 addressing

Thread devices use IPv6 addresses to identify communication endpoints. Mesh-Local addresses are used within the Thread network, while Link-Local addresses support communication on a local link. A Border Router can provide connectivity between the Thread network and other IP networks.

Border Router

A Border Router connects a Thread network to another IP network, such as Wi-Fi or Ethernet. It allows communication between the networks while Thread devices continue using IPv6.

CoAP

The Constrained Application Protocol is a lightweight application protocol designed for constrained devices. It commonly runs over UDP and uses a REST-like model: clients send requests to resources, and servers return responses.

CoAP methods

GET requests information, POST submits data or asks a resource to perform an action, PUT creates or replaces a resource's state, and DELETE removes a resource. The method's effect depends on how the application defines each resource.

CoAP message types

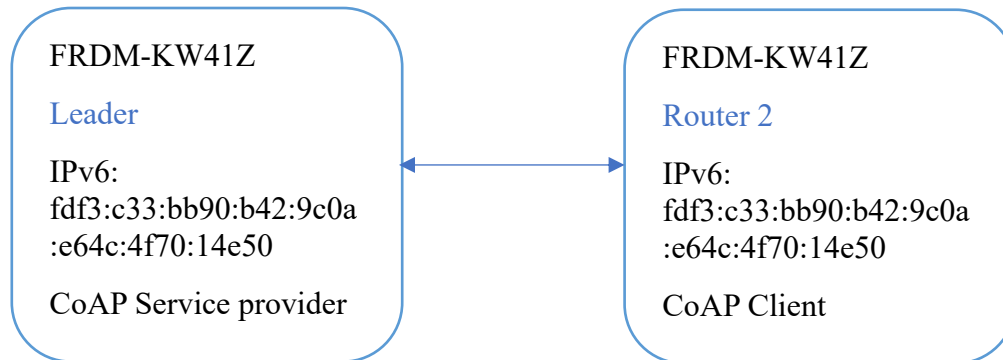
Confirmable (CON) messages require an acknowledgment and can be retransmitted if the acknowledgment does not arrive. Non-confirmable (NON) messages do not require acknowledgment, so they have less overhead but do not provide the same delivery assurance.

Multicast communication

IPv6 multicast lets a device send one message to a group of nodes rather than sending separate copies to each one. It can be useful for discovery or group communication, though the application still needs to handle replies and identify which device sent them.

Detailed network topology of the final setup

The final setup consists of two FRDM-KW41Z boards connected to the same Thread mesh network. One board operates as the Leader and provides the application services. The second board, Router2, joins the network and communicates with the Leader using CoAP over IPv6 and UDP.



Router2 sends CoAP requests to the Leader to retrieve application data. The Leader responds over the same Thread network. Thread handles IPv6 packet delivery between the devices; CoAP defines the application-level requests and responses.

Thread Network Settings

Setting	Value
PAN ID	0x7777
Channel	11
Network name	Team7

Development

Modifications made

```
80 // ROUTING ALL SINK_URI_PATH
81 #define APP_TEAM_URI_PATH "/team7"
82 #define APP_STOP_TMR_URI_PATH "/stopTmr"
83 #define APP_START_TMR_URI_PATH "/startTmr"
84 #define APP_RESTART_TMR_URI_PATH "/restartTmr"
85 #define APP_CHANGE_TIME_URI_PATH "/changeTime"
86 #define APP_ACCEL_URI_PATH "/accel"
87 #define APP_STOP_MY_TMR_URI_PATH "/stopMyTmr"
88 /* 1 = initialize the accelerometer on this board (only the Le

179 static void APP_CoapTeamCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
180 static void APP_CoapStopTimerCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
181 static void APP_CoapStartTimerCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
182 static void APP_CoapRestartTimerCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
183 static void APP_CoapChangeTimeCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
184 static void APP_TeamTimerCallback(void *param);
185 static void APP_TeamTick(uint8_t *pParam);
186 static void APP_StartTeamTimer(void);
187 static void APP_StopTeamTimer(void);
188 static void APP_CoapAccelCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
189 static void APP_AccelInit(void);
190 static bool_t APP_AccelRead(int16_t *pX, int16_t *pY, int16_t *pZ);
191 /* Router2 side */
192 static bool_t APP_IsLeader(void);
193 static void APP_TeamClientCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
194 static void APP_CoapTeamAckCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
195 static void APP_AccelClientCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
196 static void APP_CoapAccelAckCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
197 static void APP_CoapStopMyTmrCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
198 static void APP_ReqTimerCallback(void *param);
199 static void APP_ReqTimerTick(uint8_t *pParam);
200 static void APP_StartReqTimer(void);
201 static void APP_StopReqTimer(void);
202 static void APP_SendLeaderRequest(bool_t isAccel, bool_t useCon);
203 static void APP_LearnLeaderAddr(coapSession_t *pSession);
204 static void APP_ProcessAccelPayload(uint8_t *pData, uint32_t dataLen, const char *addrStr, const char *typeStr);
205 static bool_t APP_ParseAxis(const char *pStr, char axis, int16_t *pValue);
206 static void APP_AccelToLed(int16_t x, int16_t y, int16_t z);
207 static void App_RestoreLeaderLed(uint8_t *param);

---
217 Public global variables declarations
218 \=====*/
219 const coapUriPath_t gAPP_LED_URI_PATH = {SizeOfString(APP_LED_URI_PATH), (uint8_t *)APP_LED_URI_PATH};
220 const coapUriPath_t gAPP_TEMP_URI_PATH = {SizeOfString(APP_TEMP_URI_PATH), (uint8_t *)APP_TEMP_URI_PATH};
221 const coapUriPath_t gAPP_SINK_URI_PATH = {SizeOfString(APP_SINK_URI_PATH), (uint8_t *)APP_SINK_URI_PATH};
222 const coapUriPath_t gAPP_TEAM_URI_PATH = {SizeOfString(APP_TEAM_URI_PATH), (uint8_t *)APP_TEAM_URI_PATH};
223 const coapUriPath_t gAPP_STOP_TMR_URI_PATH = {SizeOfString(APP_STOP_TMR_URI_PATH), (uint8_t *)APP_STOP_TMR_URI_PATH};
224 const coapUriPath_t gAPP_START_TMR_URI_PATH = {SizeOfString(APP_START_TMR_URI_PATH), (uint8_t *)APP_START_TMR_URI_PATH};
225 const coapUriPath_t gAPP_RESTART_TMR_URI_PATH = {SizeOfString(APP_RESTART_TMR_URI_PATH), (uint8_t *)APP_RESTART_TMR_URI_PATH};
226 const coapUriPath_t gAPP_CHANGE_TIME_URI_PATH = {SizeOfString(APP_CHANGE_TIME_URI_PATH), (uint8_t *)APP_CHANGE_TIME_URI_PATH};
227 const coapUriPath_t gAPP_ACCEL_URI_PATH = {SizeOfString(APP_ACCEL_URI_PATH), (uint8_t *)APP_ACCEL_URI_PATH};
228 const coapUriPath_t gAPP_STOP_MY_TMR_URI_PATH = {SizeOfString(APP_STOP_MY_TMR_URI_PATH), (uint8_t *)APP_STOP_MY_TMR_URI_PATH};

/* Application timer Id */
tmrTimerID_t mAppTimerId = gTmrInvalidTimerID_c;
static tmrTimerID_t gTeamTimerId = gTmrInvalidTimerID_c;
static uint16_t gTeamCounter = 0;
static uint32_t gTeamTimerSeconds = 1;
static bool_t gTeamTimerRunning = FALSE;
/* Router2: timer that requests the leader's counter (and accelerometer) */
static tmrTimerID_t gReqTimerId = gTmrInvalidTimerID_c;
static bool_t gReqTimerRunning = FALSE;
static bool_t gReqUseCon = TRUE;
/* Router2: address of the leader (learned from its answers, or fixed with APP_USE_FIXED_LEADER) */
static ipAddr_t gLeaderAddr;
static bool_t gLeaderKnown = FALSE;

---
401 /* Router2: when connected, start requesting the leader's counter every APP_REQUEST_SECONDS
402 (the tick does nothing if this device is the Leader) */
403 gReqUseCon = TRUE;
404 APP_StartReqTimer();
```

```

426      /* Start Team 7 counter (1 to 150, 1 second) */
427      gTeamCounter = 0;
428      gTeamTimerSeconds = 1;
429      APP_StartTeamTimer();

```

Here we added registrations for the new URI paths because the CoAP server must associate each request path with its corresponding callback.

```

static void APP_InitCoapDemo
(
    void
)
{
    coapRegCbParams_t cbParams[] = {
        {APP_CoapLedCb, (coapUriPath_t *)&gAPP_LED_URI_PATH},
        {APP_CoapTempCb, (coapUriPath_t *)&gAPP_TEMP_URI_PATH},
#ifdef LARGE_NETWORK
        {APP_CoapResetToFactoryDefaultsCb, (coapUriPath_t *)&gAPP_RESET_URI_PATH},
#endif
        {APP_CoapSinkCb, (coapUriPath_t *)&gAPP_SINK_URI_PATH},
        {APP_CoapTeamCb, (coapUriPath_t *)&gAPP_TEAM_URI_PATH},
        {APP_CoapStopTimerCb, (coapUriPath_t *)&gAPP_STOP_TMR_URI_PATH},
        {APP_CoapStartTimerCb, (coapUriPath_t *)&gAPP_START_TMR_URI_PATH},
        {APP_CoapRestartTimerCb, (coapUriPath_t *)&gAPP_RESTART_TMR_URI_PATH},
        {APP_CoapChangeTimeCb, (coapUriPath_t *)&gAPP_CHANGE_TIME_URI_PATH},
        {APP_CoapAccelCb, (coapUriPath_t *)&gAPP_ACCEL_URI_PATH},
        {APP_CoapStopMyTmrCb, (coapUriPath_t *)&gAPP_STOP_MY_TMR_URI_PATH};

    /* Register Services in COAP */
    sockAddrStorage_t coapParams = {0};
    NWKU_SetSockAddrInfo(&coapParams, NULL, AF_INET6, COAP_DEFAULT_PORT, 0, gIpIfSlp0_c);
    mAppCoapInstId = COAP_CreateInstance(NULL, &coapParams, (coapRegCbParams_t *)cbParams,
                                         NumberOfElements(cbParams));
}

```

Here we added the counter update because the Leader needs to increment its value periodically and wrap it back to 1 at the configured maximum.

```

1130 /* Runs in the application task (posted by the timer callback), so it is safe to touch the counter and the shell */
1131 static void APP_TeamTick(uint8_t *pParam)
1132 {
1133     (void)pParam;
1134     if(gTeamTimerRunning)
1135     {
1136         /* Counter goes from 1 to 150 and then wraps back to 1 */
1137         if(gTeamCounter >= APP_COUNTER_MAX)
1138         {
1139             gTeamCounter = 1U;
1140         }
1141         else
1142         {
1143             gTeamCounter++;
1144         }
1145         #if APP_TEAM_PRINT_TICK
1146             shell_printf("\rCounter = %u\r\n", gTeamCounter);
1147         #endif
1148     }
1149 }

```

```

1229 static void APP_CoapStopTimerCb
1230 {
1231     coapSessionStatus_t sessionStatus,
1232     uint8_t *pData,
1233     coapSession_t *pSession,
1234     uint32_t dataLen
1235 }
1236 {
1237     char addrStr[INET6_ADDRSTRLEN];
1238     (void)pData;
1239     (void)dataLen;
1240     if(sessionStatus != gCoapFailure_c)
1241     {
1242         ntop(AF_INET6,
1243             (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr,
1244             addrStr,
1245             INET6_ADDRSTRLEN);
1246         APP_StopTeamTimer();
1247         if(pSession->msgType == gCoapConfirmable_c)
1248         {
1249             shell_printf("\rTimer stopped from %s type CON Count= %u\r\n", addrStr, gTeamCounter);
1250             COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
1251         }
1252         else if(pSession->msgType == gCoapNonConfirmable_c)
1253         {
1254             shell_printf("\rTimer stopped from %s type NON Count= %u\r\n", addrStr, gTeamCounter);
1255         }
1256     }
1257 }

1258 static void APP_CoapStartTimerCb
1259 {
1260     coapSessionStatus_t sessionStatus,
1261     uint8_t *pData,
1262     coapSession_t *pSession,
1263     uint32_t dataLen
1264 }
1265 {
1266     char addrStr[INET6_ADDRSTRLEN];
1267     (void)pData;
1268     (void)dataLen;
1269     if(sessionStatus != gCoapFailure_c)
1270     {
1271         ntop(AF_INET6,
1272             (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr,
1273             addrStr,
1274             INET6_ADDRSTRLEN);
1275         APP_StopTeamTimer();
1276         gTeamCounter = 0;
1277         APP_StartTeamTimer();
1278         if(pSession->msgType == gCoapConfirmable_c)
1279         {
1280             shell_printf("\rTimer started from %s type CON starting Count= %u\r\n", addrStr, gTeamCounter);
1281             COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
1282         }
1283         else if(pSession->msgType == gCoapNonConfirmable_c)
1284         {
1285             shell_printf("\rTimer started from %s type NON starting Count= %u\r\n", addrStr, gTeamCounter);
1286         }
1287     }

```

```

1289 static void APP_CoapRestartTimerCb
1290 {
1291     coapSessionStatus_t sessionStatus,
1292     uint8_t *pData,
1293     coapSession_t *pSession,
1294     uint32_t dataLen
1295 }
1296 {
1297     char addrStr[INET6_ADDRSTRLEN];
1298     (void)pData;
1299     (void)dataLen;
1300     if(sessionStatus != gCoapFailure_c)
1301     {
1302         ntop(AF_INET6,
1303             (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr,
1304             addrStr,
1305             INET6_ADDRSTRLEN);
1306         APP_StartTeamTimer();
1307         if(pSession->msgType == gCoapConfirmable_c)
1308         {
1309             shell_printf("\rTimer started from %s type CON starting Count= %u\r\n", addrStr, gTeamCounter);
1310             COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
1311         }
1312         else if(pSession->msgType == gCoapNonConfirmable_c)
1313         {
1314             shell_printf("\rTimer started from %s type NON starting Count= %u\r\n", addrStr, gTeamCounter);
1315         }
1316     }
1317 }

1318 static void APP_CoapChangeTimeCb
1319 {
1320     coapSessionStatus_t sessionStatus,
1321     uint8_t *pData,
1322     coapSession_t *pSession,
1323     uint32_t dataLen
1324 }
1325 {
1326     char addrStr[INET6_ADDRSTRLEN];
1327     char timeString[12] = {0};
1328     uint32_t newTime = 0;
1329     uint32_t copyLen;
1330     if((sessionStatus != gCoapFailure_c) && (pData != NULL) && (dataLen != 0U))
1331     {
1332         ntop(AF_INET6,
1333             (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr,
1334             addrStr,
1335             INET6_ADDRSTRLEN);
1336         copyLen = (dataLen < (sizeof(timeString) - 1U)) ? dataLen : (sizeof(timeString) - 1U);
1337         FLlib_MemCpy(timeString, pData, copyLen);
1338         newTime = APP_StringToUint32((uint8_t *)timeString, copyLen);
1339         if((newTime > 0U) && (newTime <= APP_TIMER_SECONDS_MAX))
1340         {
1341             gTeamTimerSeconds = newTime;
1342             if(gTeamTimerRunning)
1343             {
1344                 APP_StopTeamTimer();
1345                 APP_StartTeamTimer();
1346             }
1347             if(pSession->msgType == gCoapConfirmable_c)

```

```

1348         {
1349             shell_printf("\rTimer changed to %u seconds from %s type CON\r\n", gTeamTimerSeconds, addrStr);
1350         }
1351         else if(pSession->msgType == gCoapNonConfirmable_c)
1352         {
1353             shell_printf("\rTimer changed to %u seconds from %s type NON\r\n", gTeamTimerSeconds, addrStr);
1354         }
1355     }
1356     else
1357     {
1358         shell_printf("\rInvalid time value (1 to %u seconds)\r\n", APP_TIMER_SECONDS_MAX);
1359     }
1360     /* A CON request is always answered with an ACK, even if the value was invalid */
1361     if(pSession->msgType == gCoapConfirmable_c)
1362     {
1363         COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
1364     }
1365 }
1366 }

1468 static void APP_CoapStopMyTmrCb
1469 {
1470     coapSessionStatus_t sessionStatus,
1471     uint8_t *pData,
1472     coapSession_t *pSession,
1473     uint32_t dataLen
1474 }
1475 {
1476     char addrStr[INET6_ADDRSTRLEN];
1477     (void)pData;
1478     (void)dataLen;
1479     if(sessionStatus != gCoapFailure_c)
1480     {
1481         ntop(AF_INET6,
1482             (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr,
1483             addrStr,
1484             INET6_ADDRSTRLEN);
1485         APP_StopReqTimer();
1486         if(pSession->msgType == gCoapConfirmable_c)
1487         {
1488             shell_printf("\rRequest timer stopped from %s type CON\r\n", addrStr);
1489             COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
1490         }
1491         else if(pSession->msgType == gCoapNonConfirmable_c)
1492         {
1493             shell_printf("\rRequest timer stopped from %s type NON\r\n", addrStr);
1494         }
1495     }
1496 }

```

```

1634 static void APP_AccelToLed(int16_t x, int16_t y, int16_t z)
1635 {
1636     int32_t ax = (x < 0) ? -(int32_t)x : (int32_t)x;
1637     int32_t ay = (y < 0) ? -(int32_t)y : (int32_t)y;
1638     int32_t az = (z < 0) ? -(int32_t)z : (int32_t)z;
1639     int32_t max = ax;
1640     uint8_t redValue = 0, greenValue = 0, blueValue = 0;
1641     if(ay > max){
1642         max = ay;
1643     }
1644     if(az > max){
1645         max = az;
1646     }
1647     if(ax >= (max - APP_ACCEL_TOLERANCE)){
1648         /* X -> green */
1649         greenValue = 255;
1650     }
1651     if(ay >= (max - APP_ACCEL_TOLERANCE)){
1652         /* Y -> magenta (red + blue) */
1653         redValue = 255;
1654         blueValue = 255;
1655     }
1656     if(az >= (max - APP_ACCEL_TOLERANCE)){
1657         /* Z -> cyan (green + blue) */
1658         greenValue = 255;
1659         blueValue = 255;
1660     }
1661     APP_SetMode(mThrInstanceId, gDeviceMode_Application_c);

```

```

1870 static void APP_CoapAccelCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen){
1871     char addrStr[INET6_ADDRSTRLEN];
1872     char payload[32];
1873     int16_t x = 0, y = 0, z = 0;
1874     uint32_t payloadLen;
1875     (void)pData;
1876     (void)dataLen;
1877     /* Only the leader answers with its accelerometer. On the other device this URI receives the NON answers */
1878     if(!APP_IsLeader()){
1879         APP_AccelClientCb(sessionStatus, pData, pSession, dataLen);
1880         return;
1881     }
1882     if(sessionStatus != gCoapFailure_c){
1883         ntop(AF_INET6,
1884             (ipAddr_t *) &pSession->remoteAddrStorage.ss_addr,
1885             addrStr,
1886             INET6_ADDRSTRLEN);
1887         if(!APP_AccelRead(&x, &y, &z))
1888         {
1889             shell_printf("\rAccelerometer read error\r\n");
1890         }
1891         sprintf(payload, "X=%d Y=%d Z=%d", x, y, z);
1892         payloadLen = strlen(payload);
1893         if(pSession->msgType == gCoapConfirmable_c)
1894         {
1895             shell_printf("\rCON accel request received from %s\r\n", addrStr);
1896             COAP_Send(pSession, gCoapMsgTypeAckSuccessContent_c, (uint8_t *)payload, payloadLen);
1897         }
1898         else if(pSession->msgType == gCoapNonConfirmable_c)
1899         {
1900             shell_printf("\rNON accel request received from %s\r\n", addrStr);
1901             pSession->pUriPath = (coapUriPath_t *) &gAPP_ACCEL_URI_PATH;
1902             COAP_Send(pSession, gCoapMsgTypeNonPost_c, (uint8_t *)payload, payloadLen);
1903         }

```

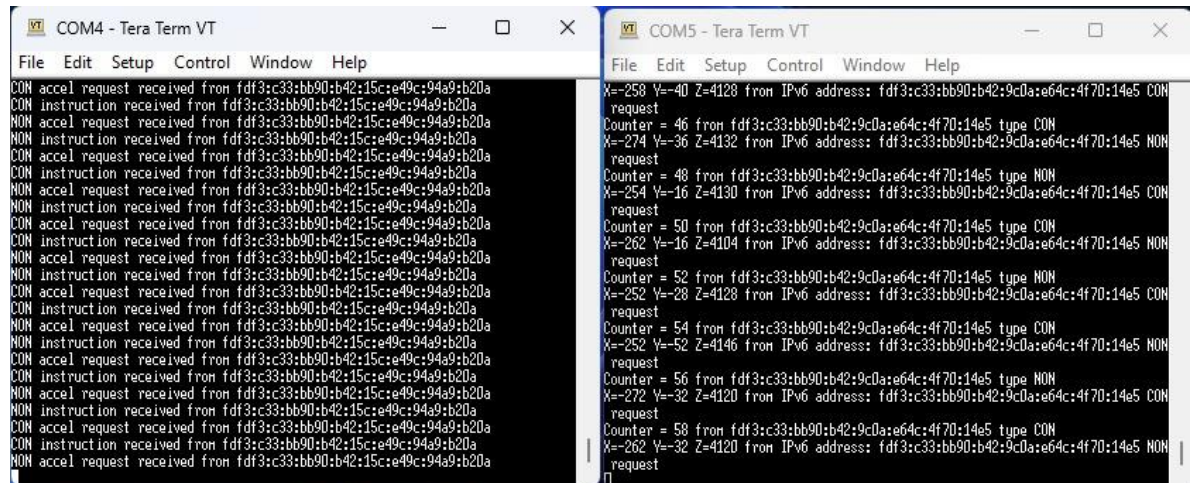
```

1906 static void APP_CoapLedCb
1907 {
1908     coapSessionStatus_t sessionStatus,
1909     uint8_t *pData,
1910     coapSession_t *pSession,
1911     uint32_t dataLen
1912 }
1913 {
1914     /* Process the command only if it is a POST method */
1915     if((pData) && (sessionStatus == gCoapSuccess_c) && (pSession->code == gCoapPOST_c))
1916     {
1917         APP_ProcessLedCmd(pData, dataLen);
1918     }
1919     /* Send the reply if the status is Success or Duplicate */
1920     if((gCoapFailure_c != sessionStatus) && (gCoapConfirmable_c == pSession->msgType))
1921     {
1922         /* Send CoAP ACK */
1923         COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
1924     }
1925 }

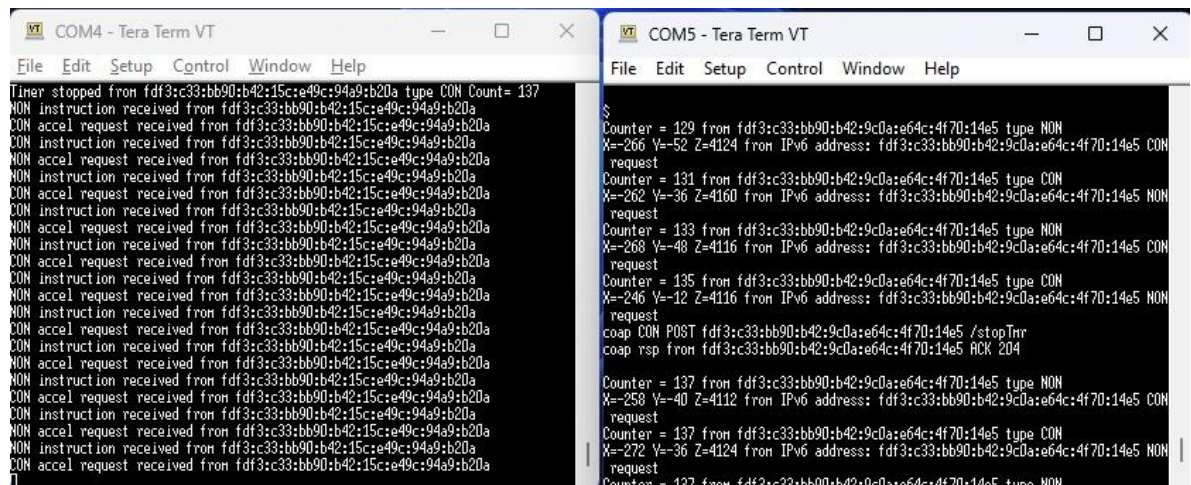
```

Screenshots

- Request timer



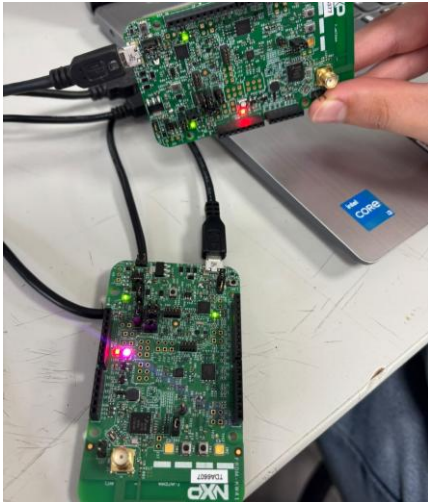
- CoAP stopTmr



- Highest axis: X



- Highest axis: Y



- Highest axis: Z



Conclusions

Sofia Davis: What makes Thread useful is that it combines mesh networking with standard IPv6, so devices can exchange data without needing a full Wi-Fi-style setup. The process was not completely smooth: interpreting serial output and distinguishing CON requests from NON requests took time, and a message that did not produce the expected result was not always easy to diagnose immediately. I would describe the implementation as manageable rather than easy. The SDK handled much of the networking underneath, but we still needed to understand the callbacks, timers, and message flow to connect the application behavior correctly.

Cuauhtemoc Herrera: Thread seems like a practical choice for networks with sensors and other small devices because it handles network roles and routing while using wireless communication designed for low-power devices. The trickiest part for me was testing the behavior between the boards, especially understanding where a request was going and what kind of response it should receive. For example, the `/stopMyTmr` URI affects Router2's own request timer, which is different from stopping the Leader's counter timer. Implementing the feature took some trial and error, but it became much clearer after checking the URI behavior directly from the terminal.

Plinio Yadin: I think Thread is useful for IoT because it lets low-power devices communicate using IPv6 and pass messages across a mesh instead of relying on every device to connect directly to one central point. During the project, one challenge was figuring out whether a communication problem came from the Thread connection, the IPv6 address, or the CoAP URI being used. Testing commands from the serial terminal helped us separate those possibilities. The implementation was not especially easy at first, but the existing SDK gave us a solid starting point once we understood how the pieces worked together.